

OneCompiler

Upgrade

AI Java Run

Save Login

886 ms

EXPLORER

Main.java

```
1 import java.util.HashMap;
2 import java.util.Map;
3
4 public class Main {
5     static final String ALUNO = "PEDRO LUIZ CARVALHO MENDES";
6     static final String RA = "24055157-2";
7
8     static final String ENTRADA = "total_PLCM = valor + 72;";
9
10    private static final Map<Character, String> SIMBOLOS = new HashMap<>();
11
12    static {
13        SIMBOLOS.put('=', "ATRIBUICAO");
14        SIMBOLOS.put('+', "SOMA");
15        SIMBOLOS.put('-', "SUBTRACAO");
16        SIMBOLOS.put('*', "MULTIPLICACAO");
17        SIMBOLOS.put('/', "DIVISAO");
18        SIMBOLOS.put('.', "PONTO_E_VIRGULA");
19        SIMBOLOS.put('(', "ABRE_PARENTESES");
20        SIMBOLOS.put(')', "FECHA_PARENTESES");
21    }
22
23    public static void main(String[] args) {
24        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
25        System.out.println("ENTRADA: " + ENTRADA);
26        System.out.println();
27        System.out.printf("%-22s %s\n", "LEXEMA", "TOKEN");
28        System.out.println("-----");
```

Console

I/O

AI Agent

ALUNO: PEDRO LUIZ CARVALHO MENDES | RA: 24055157-2
ENTRADA: total_PLCM = valor + 72;

LEXEMA	TOKEN
total_PLCM	IDENTIFICADOR
=	ATRIBUICAO
valor	IDENTIFICADOR
+	SOMA
72	INTEIRO
;	PONTO_E_VIRGULA

Ready

Terminal

Light Wiki Java

OneCompiler

Upgrade

AI Java Run

Save Login

1.3s

Main.java

```
4
5 public class Main {
6     static final String ALUNO = "PEDRO LUIZ CARVALHO MENDES";
7     static final String RA = "24055157-2";
8
9     static final String IDENTIFICADOR_PERSONALIZADO = "PLCM";
10
11    static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>()
12    {
13        Arrays.asList("int", "if", "else", "while", "return")
14    };
15
16    public static void main(String[] args) {
17        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
18        executarTeste("TESTE A - entrada compacta",
19            "int total=valor+10;");
20        executarTeste("TESTE B - espaços e comentário",
21            "int total = valor + 10; // este comentario sera ig");
22        executarTeste("TESTE C - reservadas e identificadores",
23            "int inteiro = 0; while (inteiro < intValue) inteiro = i");
24        executarTeste("TESTE C2 - identificador personalizado",
25            "int " + IDENTIFICADOR_PERSONALIZADO + " = 10;");
26        executarTeste("TESTE D - caractere inválido",
27            "int valor# = 1;");
28    }
29
30    static void executarTeste(String titulo, String entrada) {
31        System.out.println();
32        System.out.println("=== " + titulo + " ===");
33    }
34}
```

Console

I/O

AI Agent

inteiro	LETRA	IDENTIFICADOR
+	SIMBOLO	SOMA
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE C2 - identificador personalizado ===
ENTRADA: int PLCM = 10;
LEXEMA CLASSE TOKEN

int	LETRA	PALAVRA_RESERVADA
PLCM	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
10	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE D - caractere inválido ===
ENTRADA: int valor# = 1;
LEXEMA CLASSE TOKEN

int	LETRA	PALAVRA_RESERVADA
valor	LETRA	IDENTIFICADOR
#	DESCONHECIDO	ERRO_LEXICO
=	SIMBOLO	ATRIBUICAO
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

Ready

Terminal

Light Wiki Java

OneCompiler

Upgrade

AI Java Run

Save Login

Exit 1

Main.java

```
1 public class Main {
2     static final String ALUNO = "PEDRO LUIZ CARVALHO MENDES";
3     static final String RA = "24055157-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
16 }
```

Console

I/O

AI Agent

Main.java:8: error: ';' expected
int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
^

Main.java:10: error: ')' expected
if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
^

Main.java:11: error: unclosed string literal
System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
^

3 errors

Debug this error

OneCompiler

Upgrade

AI

Java

Run

Save

Login

Main.java

```
1 public class Main {
2     static final String ALUNO = "PEDRO LUIZ CARVALHO MENDES";
3     static final String RA = "24055157-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20; // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20) { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
16
```

Console

I/O

AI Agent

770 ms

ALUNO: PEDRO LUIZ CARVALHO MENDES | RA: 24055157-2

Programa corrigido!

Resultado: 30

Ready

Terminal

Light

Wiki

Java

OneCompilerUpgrade

AIJavaRun

SaveLogin

JMain.java

```
1 public class Main {
2     static final String ALUNO = "PEDRO LUIZ CARVALHO MENDES";
3     static final String RA = "24055157-2";
4
5     static final String ENTRADA = "(2 + 3) * 4";
6     static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;
7
8     public static void main(String[] args) {
9         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
10        System.out.println("ENTRADA: " + ENTRADA);
11        System.out.println();
12
13        if (DEMONSTRAR_RECURSAO_ESQUERDA) {
14            demonstrarRecursoEsquerda(0);
15            System.out.println();
16        }
17
18        try {
19            Parser parser = new Parser(ENTRADA);
20            int resultado = parser.analisar();
21            System.out.println();
22            System.out.println("ENTRADA ACEITA");
23            System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
24        } catch (ErroSintatico erro) {
25            System.out.println();
26            System.out.println("ERRO SINTATICO: " + erro.getMessage());
27        }
28    }
29 }
```

Console

I/OAI Agent1.0s

```
SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead: )
SAINDO de <factor> | lookahead = )
SAINDO de <term> | lookahead = )
SAINDO de <expr> | lookahead = )
Novo lookahead: *
Fechando parenteses
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 4
ENTRANDO em <factor> | lookahead = 4
Numero reconhecido: 4
Novo lookahead: $FIM$
SAINDO de <factor> | lookahead = $FIM$
SAINDO de <term> | lookahead = $FIM$
SAINDO de <expr> | lookahead = $FIM$

ENTRADA ACEITA
RESULTADO DA EXPRESSAO: 20
```

Ready

Terminal

LightWikiJava

OneCompilerUpgrade

AIJavaRun

SaveLogin

JMain.java

```
1 import java.util.ArrayList;
2 import java.util.HashMap;
3 import java.util.List;
4 import java.util.Map;
5
6 public class Main {
7     static final String ALUNO = "PEDRO LUIZ CARVALHO MENDES";
8     static final String RA = "24055157-2";
9
10    static final String ENTRADA_VALIDA = "id*id*id";
11    static final String ENTRADA_INVALIDA = "id*id";
12
13    // Na primeira execucao, use ENTRADA_VALIDA. Depois troque por ENTRADA_INVALIDA.
14    static final String ENTRADA = ENTRADA_VALIDA;
15
16    static final Map<Integer, Map<String, String>> ACTION = new HashMap<>();
17    static final Map<Integer, Map<String, Integer>> GOTO = new HashMap<>();
18    static final Map<Integer, Regra> REGRAS = new HashMap<>();
19
20    static {
21        // Gramatica:
22        // 1) E -> E + T    2) E -> T
23        // 3) T -> T * F    4) T -> F
24        // 5) F -> ( E )    6) F -> id
25        regra(1, "E", 3, "E -> E + T");
26        regra(2, "E", 1, "E -> T");
27        regra(3, "T", 3, "T -> T * F");
28        regra(4, "T", 1, "T -> F");
29    }
30 }
```

Console

I/OAI Agent1.4s

```
$ 0 id 5 +id*id$ REDUCE
F -> id
$ 0 F 3 +id*id$ REDUCE
T -> F
$ 0 T 2 +id*id$ REDUCE
E -> T
$ 0 E 1 +id*id$ SHIFT
+ -> estado 6
$ 0 E 1 + 6 id*id$ SHIFT
id -> estado 5
$ 0 E 1 + 6 id 5 *id$ REDUCE
F -> id
$ 0 E 1 + 6 F 3 *id$ REDUCE
T -> F
$ 0 E 1 + 6 T 9 *id$ SHIFT
* -> estado 7
$ 0 E 1 + 6 T 9 * 7 id$ SHIFT
id -> estado 5
$ 0 E 1 + 6 T 9 * 7 id 5 $ REDUCE
F -> id
$ 0 E 1 + 6 T 9 * 7 F 10 $ REDUCE
T -> T * F
$ 0 E 1 + 6 T 9 $ REDUCE
E -> E + T
$ 0 E 1 $ ACCEPT
```

Ready

Terminal

LightWikiJava